Listing 2.5. Computing the best action

```
def get_best_arm(record):
    arm_index = np.argmax(record[:,1],axis=0)     1
    return arm_index
```

- ***1*** Uses numpy argmax on column 1 of the record array

Now we can get into the main loop for playing the *n*-armed bandit game. If a random number is greater than the epsilon parameter, we just calculate the best action using the `get_best_arm` function and take that action. Otherwise we take a random action to ensure some amount of exploration. After choosing the arm, we use the `get_reward` function and observe the reward value. We then update the `record` array with this new observation. We repeat this process a bunch of times, and it will continually update the *record* array. The arm with the highest reward probability should eventually get chosen most often, since it will give out the highest average reward.

We've set it to play 500 times in the following listing, and to display a `matplotlib` scatter plot of the mean reward against plays. Hopefully we'll see that the mean reward increases as we play more times.

Listing 2.6. Solving the *n*-armed bandit

```
fig, ax = plt.subplots(1,1)
ax.set_xlabel("Plays")
ax.set_ylabel("Avg Reward")
record = np.zeros((n,2))                              1
probs = np.random.rand(n)                             2
eps = 0.2
rewards = [0]
for i in range(500):
    if random.random() > eps:                             3
        choice = get_best_arm(record)
    else:
        choice = np.random.randint(10)
    r = get_reward(probs[choice])                     4
    record = update_record(record,choice,r)           5
    mean_reward = ((i+1) * rewards[-1] + r)/(i+2)     6
    rewards.append(mean_reward)
ax.scatter(np.arange(len(rewards)),rewards)
```

- ***1*** Initializes the record array to all zeros
- ***2*** Randomly initializes the probabilities of rewards for each arm